

Как устроена игра: архитектура и функции

Учебный путеводитель по текущему коду `src/main/kotlin`. Описаны все объявленные функции рабочего приложения, включая приватные вспомогательные методы, методы интерфейсов, точки входа и значимые блоки `init`. В конце отдельно объяснены проверки в `src/test/kotlin`. Названия функций и классов соответствуют исходникам на момент составления документа.

1. Сначала общая картина

У приложения **две точки входа**. `main.kt` запускает текстовую версию, а `game/ui/DesktopMain.kt` — графическую. Они создают разные способы общения с пользователем, но пользуются одним игровым ядром и одним типом хранилища.

Консоль: `main` → `Console` → `GameAdmin` → `Game` → `Round` / `CardEffectHandler`
└─→ `GameRepository` → `SQLite`

GUI: `DesktopMain` → `DesktopWindow` → `DesktopGameController` → `Game`
└─→ `GameRepository` → `SQLite`
└─→ `Round`
└─→ `CardEffectHandler`

`game.domain` — модель игры: игроки, карты, колода, ход, раунд, правила и очки. Она не знает о Swing, консоли или SQL. `game.ui` — два интерфейса для человека: `Console` и `DesktopWindow`. `game.logic` — консольный сценарий `GameAdmin` и контракты ввода/вывода. `game.data` — договор `GameRepository`, реализация на `SQLite` и расчёт статистики.

В GUI схема **близка к MVC**: `Game` и остальные классы `domain` — Model, `DesktopWindow` — View, `DesktopGameController` — Controller. Это не строгая учебниковая MVC: окно местами читает `controller.game` и собирает `MoveDetails` само. Экран обновляется вручную вызовом `render()`, наблюдаемых потоков состояния или автоматической привязки данных здесь нет. Для небольшой Swing-программы это допустимый способ.

Словарь, без которого сложно читать функции

- `Move` — запрос на ход: кто играет, в кого, какую карту и какие дополнительные решения приняли игроки.
- `MoveDetails` — дополнительные данные конкретной карты, например выбранные карты для обмена.
- `MoveResult` — результат применения эффекта внутри раунда.
- `GameMoveResult` — ответ всей игры вызывающему коду: отказ, обычный ход или конец раунда.
- `MoveEvent` — событие, которое интерфейс должен показать, например открытые карты.
- `GameResult` — итог завершённой партии: победители, очки, журнал.
- `Player.UID` — номер игрока **внутри партии**; постоянная запись игрока в `SQLite` связана с именем.

- `null` в проверках хода обычно значит «ошибки нет», а в диалогах выбора — «пользователь отменил действие». Значение зависит от функции.

Один ход от кнопки до базы

1. `DesktopWindow.renderGame()` показывает руку и создаёт кнопку для каждой карты.
2. Кнопка вызывает `DesktopWindow.play(index)`. Окно спрашивает цель и дополнительные карты через диалоги.
3. `DesktopGameController.submitMove(...)` создаёт `Move` и вызывает `Game.processGameMove(move)`.
4. `Game` сначала спрашивает `Round.checkMove`, потом `CardEffectHandler.validate`, затем `CardEffectHandler.apply`.
5. Если эффект применён, `Round.recordMove` записывает ход и передвигает очередь либо завершает раунд.
6. `Game` начисляет очки и возвращает `GameMoveResult`.
7. Контроллер при окончании партии вызывает `Game.getResult()`, сохраняет его через `GameRepository.writeGame()` и обновляет статистику.
8. `DesktopWindow` показывает событие, а общая обёртка `perform()` вызывает `render()`, чтобы заново собрать экран.

При запуске из консоли шаги 3–6 те же. Только ход собирает `Console.inputMove()`, а результат обрабатывает `GameAdmin.runGame()` через методы `Output`.

2. Точки входа и настройка GUI

`main.kt` — `main()`

Что делает. Создает `Console` и `SqliteGameRepository`, берёт путь к базе из `BOARD_GAME_DB` или использует `data/board_game.db`, передаёт зависимости в `GameAdmin` и вызывает `runGame()`. Ошибку запуска печатает вместе со стеком вызовов. **Зачем здесь.** Это точка сборки консольной программы; ни правила, ни SQL-запросы в ней не выполняются.

`DesktopMain.kt` — `main()`

Что делает. Получает путь к базе, передаёт запуск `Swing` в `SwingUtilities.invokeLater`, затем создаёт репозиторий, `DesktopGameController` и `DesktopWindow`. До создания компонентов вызывает `configureSwingAppearance()`, после создания делает окно видимым. При ошибке запуска показывает `JOptionPane`. **Зачем здесь.** Это отдельная точка сборки GUI. `invokeLater` запускает работу с `Swing` в его потоке обработки событий; создание окна не смешано с игровыми правилами.

`DesktopMain.kt` — `configureSwingAppearance()`

Что делает. Подбирает размер шрифта по высоте экрана, записывает шрифт для нескольких видов компонентов в глобальный `UIManager` и увеличивает отступы вкладок. **Зачем здесь.** Это настройка всего интерфейса до создания окна, поэтому она находится

рядом с запуском GUI, а не в Game, контроллере или отдельной вкладке. Функция private: остальному проекту её вызывать не нужно.

3. Графический контроллер — DesktopGameController

Роль класса. Хранит состояние текущей графической сессии: зарегистрированных игроков, выбранный состав, активную игру, видимость руки, историю и ID сохранённой партии. В конструктор получает GameRepository; gameFactory позволяет подменить создание Game в тесте.

```
init { refresh() }
```

При создании контроллера сразу загружает реестр/статистику и историю из базы. Благодаря этому стартовое окно показывает уже сохранённых игроков. Это инициализация состояния сессии, а не правило игры.

```
refresh()
```

Вызывает repository.getStats() и repository.getHistory(), обновляет свойства players и history. Окно затем читает эти свойства при render(). Контроллеру принадлежит связь с хранилищем, поэтому DesktopWindow не выполняет запросы к SQLite.

```
addPlayer(name)
```

Проверяет, что партия ещё не началась, состав не заполнен, имя после trim() не пустое и такого участника ещё нет в текущем составе. Ищет имя среди зарегистрированных игроков без учёта регистра. Если имени нет, вызывает repository.registerPlayer(clean) и refresh(); затем добавляет имя в selectedNames. Важное различие: регистрация сохраняет профиль в базе, а выбор имени добавляет человека только в состав новой партии.

```
removePlayer(name)
```

Пока игры нет, удаляет имя из selectedNames. Запись игрока в базе при этом **не удаляется**: человек может снова участвовать в следующей партии, а его статистика остаётся. Это операция над составом текущей сессии.

```
startGame()
```

Проверяет отсутствие действующей партии и число участников от 3 до 8. Превращает выбранные имена в объекты Player с UID от 1, создаёт Game через gameFactory, скрывает руку и сбрасывает ID прежней сохранённой партии. Контроллер начинает сценарий GUI, а Game уже сама запускает раунд.

```
revealHand()
```

Проверяет, что игра есть и продолжается, затем ставит handVisible = true. Это состояние интерфейса: правила не меняются, но renderGame() теперь может показать карты текущего игрока.

`visibleCards()`

Возвращает копию карт текущего игрока, если рука открыта; иначе пустой список. Окно пользуется этим методом при построении кнопок карт. Таким образом контроллер хотя бы частично закрывает доступ к скрытой руке, хотя само окно всё ещё имеет доступ к game.

`hideHand()`

Устанавливает `handVisible = false`. Вызывается при уходе с игровой вкладки, отмене некоторых диалогов и после успешного хода; цель — не оставить карты видимыми для следующего игрока.

`submitMove(cardIndex, targetUid, details)`

Проверяет наличие игры и открытую руку. По индексу находит карту текущего игрока, по UID — объект цели, собирает Move и вызывает `Game.processGameMove()`. Возвращает результат игры окну. Если ход принят, скрывает руку; если партия закончилась, вызывает `repository.writeGame(current.getResult())`, запоминает ID и обновляет историю/статистику через `refresh()`. Именно здесь GUI-сценарий соединяет интерфейс, домен и сохранение; правила карт остаются в домене.

`returnToLobby()`

Разрешает возврат только после окончания партии. Сбрасывает ссылку на текущую игру, скрывает руку, очищает текущий состав и перечитывает сохранённые данные. База и прежние результаты от этого не стираются.

4. Окно — `DesktopWindow`

Роль класса. Создаёт компоненты Swing, реагирует на нажатия и показывает состояние, полученное от контроллера и игры. Конструктор принимает `DesktopGameController`; класс наследуется от `JFrame`, то есть сам является окном.

`init { ... }`

Настраивает закрытие окна, размеры и положение на экране. Создаёт вкладки «Игроки», «Игра», «Статистика», «История» и помещает панели в `JScrollPane`, чтобы длинный список прокручивался. Запрещает редактировать статистику и историю. Обработчик вкладок скрывает открытую руку при уходе с игры; обработчик Enter в поле имени добавляет участника. Последний вызов `render()` впервые заполняет окно данными.

`column(panel)`

Устанавливает для панели `BoxLayout` по вертикальной оси и отступ от краёв. Она **не создаёт игроков**; только говорит Swing располагать следующие компоненты сверху вниз. Используется при сборке лобби и игрового экрана.

`line(panel, component)`

Выравнивает переданный компонент слева, добавляет его в панель, затем добавляет небольшой вертикальный промежуток. Когда цикл `selectedNames.forEach` вызывает `line` для каждого участника, строки появляются одна под другой. Функция отвечает только за расположение.

`button(text, action)`

Создаёт `JButton`, задаёт внутренние отступы и регистрирует обработчик нажатия. Переданная лямбда `action` выполняется через `perform()`. Это уменьшает повторение одинаковых строк при создании многих кнопок.

`perform(action)`

Вызывает действие в `try`, при исключении показывает диалог с ошибкой, а в `finally` вызывает `render()`. Так после действия кнопки экран обновляется даже при ошибке. Архитектурно это общий обработчик событий `View`. Ограничение текущего кода: ловится любой `Exception`, поэтому программная ошибка тоже будет показана как обычное сообщение пользователя.

`showError(message)`

Показывает модальное окно ошибки через `Swing JOptionPane`. В `Game` и `Round` такого вызова нет: модель возвращает сообщение, а интерфейс решает, как его показать.

`addPlayerFromField()`

Передаёт содержимое `nameField` в `controller.addPlayer()` и очищает поле после успешного вызова. Окно отвечает за поле ввода, контроллер — за регистрацию и текущий состав.

`render()`

Вызывает `renderLobby()` и `renderGame()`, затем превращает `controller.players` и `controller.history` в текст двух других вкладок. `revalidate()` просит `Swing` пересчитать расположение, `repaint()` — обновить изображение окна. Это **ручная синхронизация** экрана с состоянием: после изменения модели контроллер сам не посылает наблюдаемое значение в `View`.

`renderLobby()`

Удаляет старые компоненты панели, настраивает вертикальную раскладку, показывает поле имени и кнопку «Добавить». Для каждого имени из `controller.selectedNames` создаёт строку с кнопкой «Убрать». Кнопка запуска создаёт игру через `controller.startGame()` и переводит на вкладку игры. Во время партии поле и кнопки изменения состава выключены. Вызов `removeAll()` удаляет **элементы экрана**, а не игроков из базы.

`renderGame()`

Заново строит игровую вкладку в зависимости от состояния: игры нет, игра завершена или продолжается. При завершении показывает победителей, очки и кнопку возврата в лобби. Во

время игры показывает текущего игрока и номер хода; пока рука скрыта, даёт кнопку открытия, затем создаёт кнопки карт по `visibleCards()`. Здесь View напрямую читает `controller.game` и часть доменной модели; это одна из причин называть MVC приблизительной, а не строгой.

`play(index)`

Берёт карту по её индексу в руке текущего игрока. Если карте нужна цель, открывает `choosePlayer()`. Перед выбором скрывает прежнее содержимое игровой панели, чтобы при передаче устройства другой игрок не видел руку. Затем вызывает `chooseDetails()` и отправляет готовые данные в `controller.submitMove()`. По `GameMoveResult` показывает отказ, событие или конец раунда. Эта функция ведёт **диалог пользователя о ходе**; проверка, разрешён ли ход правилами, остаётся в Game и обработчике эффектов.

`showEvent(event, actor)`

Сейчас обрабатывает событие `CardsRevealed`: показывает просьбу передать устройство и затем выбранному человеку — открытые карты. `MoveEvent` отделяет факт, возникший в модели, от способа его показа в Swing.

`choosePlayer(prompt, players)`

Открывает `JOptionPane.showInputDialog` с именами допустимых целей. При отмене возвращает `null`; при выборе находит соответствующий объект `Player`. Модель получает объект игрока или UID, а диалог остаётся в View.

`chooseCards(player, played, max)`

Готовит доступные карты игрока, при необходимости исключает уже разыгрываемую карту. Просит передать устройство, показывает `JList`, допускает выбор одной либо нескольких карт и проверяет число выбранных элементов. Возвращает список карт или `null` при отмене/неудачном выборе. Это сбор пользовательского ввода; допустимость результата по правилам дополнительно проверяет `CardEffectHandler.validate()`.

`chooseDetails(card, actor, target, players, turn)`

По виду эффекта запрашивает дополнительные решения: карту для кражи, два набора карт для обмена, карту для Пса или выбор участников для Тумана/Сплетен. Собирает соответствующий вариант `MoveDetails`. Для некоторых эффектов после третьего хода возвращает `MoveDetails.None`, не показывая диалоги. `null` означает, что сбор отменён; `MoveDetails.None` означает, что дополнительные сведения не требуются. Здесь View знает названия эффектов — ещё одна практическая, но не идеальная граница между View и доменом.

5. Консольный сценарий и контракты

`GameAdmin.runGame()`

Получает игроков через `Input.inputPlayers()`, создаёт `Game` и показывает начальное состояние через `Output`. В цикле запрашивает `Move`, вызывает `Game.processGameMove()` и по типу результата показывает ошибку, событие или конец раунда. После окончания получает `GameResult`, вызывает `GameRepository.writeGame()`, затем выводит историю и статистику. Это координатор **консольной** версии; GUI не вызывает `GameAdmin`, потому что графический интерфейс живёт по отдельным нажатиям, а не в цикле `while`.

Контракт `Input`

- `inputPlayers()` обещает вернуть список доменных `Player` для начала игры. Его реализация в `Console` читает имена из терминала.
- `inputMove(game)` обещает вернуть готовый `Move` для текущей игры. Передача `Game` позволяет узнать текущего игрока и карты. Интерфейс нужен `GameAdmin`, чтобы тот зависел от способа ввода как от договора, а не от конкретного `Console`.

Контракт `Output`

- `showGameState(game, roundNumber)` — показать, где игроки находятся в партии.
- `showGameResult(game)` — показать окончательный итог и победителей.
- `showRoundResult(status)` — показать исход раунда.
- `showMoveEvent(event)` — показать дополнительное событие хода.
- `showError(message)` — показать отказ или ошибку.
- `showGameSaved(gameId)` — сообщить номер сохранённой партии.
- `showHistory(history)` — показать сохранённые партии.
- `showStats(statistics)` — показать статистику игроков.

Все эти методы вызываются из сценария `GameAdmin`, а `Console` решает, какими строками и в каком формате ответить. `Game` ничего не печатает и не зависит от `Output`.

Реализация `Console`

- `showGameState(...)` печатает текущий раунд и общее число раундов.
- `showGameResult(game)` берёт `game.getResult()` и печатает имена победителей.
- `showRoundResult(status)` печатает статус завершённого раунда.
- `showMoveEvent(event)` печатает раскрытые карты, когда пришёл `CardsRevealed`.
- `showError(message)` печатает сообщение об отказе с пометкой «ОШИБКА».
- `showGameSaved(gameId)` печатает ID записи, возвращённый репозиторием.
- `showHistory(history)` берёт пять последних партий, преобразует время и печатает победителей.
- `showStats(statistics)` печатает число игр, побед, очки и средний счёт каждого игрока.
- `inputPlayers()` спрашивает число участников и имена, не допускает повторяющиеся имена без учёта регистра, создаёт `Player` с `UID` для текущей партии. Некорректное

число заменяет значением по умолчанию и ограничивает диапазоном 3–8.

- `inputMove(game)` печатает руку текущего игрока, читает номер карты и при необходимости цель, затем вызывает `collectMoveDetails()` и создаёт `Move`. Если номер не распознан, выбирает первый доступный элемент. Это сбор ввода, а не применение правил.
- `collectMoveDetails(card, initiator, target, allPlayers)` по эффекту запрашивает необходимые карты и собирает вариант `MoveDetails`; для остальных эффектов возвращает `None`.
- `selectCards(player, promptMsg, playedCard)` печатает доступные карты, считывает номера через пробел или запятую и возвращает найденные объекты `Card`. Уже разыгрываемую карту исключает из выбора. Возможность выбрать неверное число карт затем проверяется в домене.

6. Игровая модель: `Game`, `Round`, `Player`, `Deck`

`Game` — партия целиком

Состояние. Список игроков, текущий раунд, номер раунда, история завершённых раундов, очки и статус партии. Сейчас `TOTAL_ROUNDS = 1`.

- `init { ... }` проверяет число игроков и вызывает `startNextRound()`. Сразу после создания `Game` готова принимать ходы.
- `startNextRound()` не делает ничего после лимита раундов; иначе увеличивает номер, очищает состояния игроков через `resetForRound()`, создаёт `Deck` и `Round`, меняет статус на `IN_PROGRESS`. Это координация раундов, а не детали раздачи.
- `processGameMove(move)` — **главная функция движка**. Получает текущий `Round`; отклоняет неправильный ход через `Round.checkMove()`, затем нарушение правил карты через `effectHandler.validate()`. Вызывает `effectHandler.apply()`, после успешного эффекта — `Round.recordMove()`. Если раунд идёт дальше, возвращает `MoveApplied`; иначе добавляет журнал, начисляет очки, завершает партию или начинает следующий раунд и возвращает `RoundFinished`. Ни GUI, ни консоль не должны повторять этот порядок.
- `updateScores(round)` начисляет очки по исходу раунда. При победе монстров берёт наибольшую стоимость разыгранной игроком карты стороны монстров/друзей. При победе охотников начисляет очки игрокам, не игравшим на стороне монстров; последнему инициатору даёт дополнительный балл. Метод приватный, потому что менять счёт вправе сама партия.
- `getScores()` возвращает копию карты «игрок → очки», чтобы внешний код не изменял внутренний `scores` напрямую.
- `getResult()` доступен только после `FINISHED`; находит максимальный счёт, формирует список победителей и отдаёт `GameResult` с копиями истории и очков. Это граница между текущей игрой и сохраняемым результатом.
- `getCurrentPlayer()` делегирует запрос раунду; без запущенного раунда сообщает об ошибке состояния.
- `getAllPlayers()` возвращает копию списка участников партии.
- `getRoundNumber()` сообщает номер текущего раунда.

- `getCurrentTurn()` делегирует запрос номера хода раунду.

Round — один раунд

Состояние. Очередь игроков, стартовый игрок, номер хода, колода, история и исход раунда.

- `init { ... }` раздаёт каждому по четыре карты из `Deck`. Затем ищет владельца `SearchCard` и назначает его начальным и текущим игроком. Это правило старта раунда.
- `checkMove(move)` возвращает строку ошибки, если раунд завершён, ходит не тот игрок, карты нет в руке или обладатель «Розыска» пытается сыграть другую карту. `null` значит, что базовые условия соблюдены. Условия отдельных эффектов находятся в `CardEffectHandler`.
- `recordMove(move, result)` принимает только применённый ход и дополнительно проверяет текущее состояние. По `result.roundStatus` при необходимости завершает раунд, добавляет ход в историю; если раунд продолжается, вызывает `nextPlayer()`. Эффект карты к этому моменту уже применён обработчиком.
- `nextPlayer()` переводит индекс к следующему игроку по кругу. Когда очередь возвращается к стартовому игроку, увеличивает номер хода.
- `getCurrentTurn()` возвращает номер хода; `getCurrentPlayer()` — игрока по текущему индексу.
- `getMoveLog()` превращает полные объекты `Move` в компактные `MoveLog` для результата и базы.
- `getPlayers()` возвращает копию списка игроков раунда.
- `countPlayedCards(predicate)` подсчитывает разыгранные карты, удовлетворяющие переданному условию; например, сколько было Охотников.
- `finishWithMonsterWin()` и `finishWithHuntersWin()` меняют статус раунда на соответствующую победу. Их вызывает `recordMove()` на основании результата эффекта.

Player — участник внутри партии

Состояние. `hand` — карты в руке, `discarded` — сброшенные, `played` — разыгранные. Поля списков закрыты снаружи.

- `addCard(card)` кладёт карту в руку; используется раздачей и эффектами.
- `removeCard(card)` убирает карту из руки без добавления в историю разыгранных/сброшенных; подходит для передачи карты другому игроку.
- `playCard(card)` убирает карту из руки и записывает её как разыгранную; важно для итогового подсчёта очков.
- `discardCard(card)` убирает карту из руки и записывает в сброшенные; это иное действие, чем разыгрывание.
- `getCards()`, `getDiscarded()`, `getPlayed()` возвращают копии соответствующих списков. Внешний код не получает изменяемую коллекцию игрока.
- `hasMonsterCard()` и `hasHideCard()` отвечают на вопросы о текущей руке; используются правилами эффектов.
- `hasCard(card)` возвращает количество экземпляров такой карты в руке, а не просто `true/false`.

- `resetForRound()` очищает все три списка перед новым раундом. Вызывает `Game.startNextRound()`.

Deck — колода раунда

- `init { buildAndShuffleDeck() }` готовит колоду при создании объекта.
- `buildAndShuffleDeck()` собирает карты нужных типов, перемешивает обычные карты и подготавливает начальную раздачу: среди первых карт гарантирует монстра, Охотника, Укрытие и Розыск. Затем добавляет остальные карты. Колода знает состав карт, а `Round` знает, кому их раздать.
- `nextCard()` снимает и возвращает верхнюю карту; при пустой колоде бросает ошибку.
- `isEmpty()` сообщает, остались ли карты; правила вызывают это перед добором.

Move.toLog()

Преобразует полный `Move` со ссылками на игроков и карту в `MoveLog` из `UID` и названия карты. Такой компактный журнал можно сохранить независимо от объектов живой игры. `MoveDetails`, например конкретные карты обмена, в `MoveLog` не включены.

Карты без функций

`Card` и конкретные объекты карт в основном **объявляют свойства**: `title`, `description`, `effect`, `role`, `winPoints`, `requiresTarget`. `CardEffect` задаёт тип эффекта, по которому работает обработчик. Это важно архитектурно: данные конкретной карты находятся рядом с картой, а общие правила её применения — в `CardEffectHandler`.

Monster.canBeKilledByHunter(turn, playedHunters)

Базовая версия возвращает `true`. `KumihCard` переопределяет метод: на четвёртом ходу убить её нельзя. `MermaidCard` переопределяет метод: защита действует, пока не было разыгранного Охотника. `CardEffectHandler.applyHunter()` спрашивает сам объект монстра вместо списка проверок «если Кумихо, если Русалка». Это пример полиморфизма из ООП.

7. Правила карт — CardEffectHandler

Контракт обработчика

- `validate(move, round)` обещает вернуть `null`, если условия эффекта соблюдены, или понятную причину отказа. Проверка отделена от применения, чтобы недопустимый ход не менял руки игроков.
- `apply(move, round)` обещает применить эффект и вернуть `MoveResult`. `Game` использует этот результат, чтобы записать ход и решить, закончился ли раунд.

DefaultCardEffectHandler.validate(move, round)

Проверяет, требуется ли цель и разрешено ли выбрать себя. На четвёртом ходу пропускает

проверку дополнительных выборов для эффектов, отключённых правилами. Затем по CardEffect вызывает специализированные validateTheft, validateTrade, validateChoices, validateHound, validateWerewolf, validateMandragora или проверяет ограничение остальных монстров. Не двигает очередь и не меняет карты.

DefaultCardEffectHandler.apply(move, round)

Применяет эффект после успешной проверки. Для Пса используется особый путь: карта переходит к цели, поэтому она не записывается обычным playCard(). Для других карт сначала вызывает initiator.playCard(), затем по типу эффекта вызывает нужный apply... или создаёт событие раскрытия карт. Возвращает обычный исход либо победу стороны, но **статус самого Round меняет позже Round.recordMove()**.

Вспомогательные проверки

- applied(event) создаёт обычный MoveResult.Applied(IN_PROGRESS, event). Повторяющийся результат не приходится собирать в каждой ветке.
- validateTheft(move) требует MoveDetails.Theft и реальную карту в руке инициатора, отличную от разыгрываемой.
- validateTrade(move) требует MoveDetails.Trade, от одной до двух карт каждой стороны и одинаковое количество. Через ownsAll() проверяет, что обе стороны действительно владеют указанными картами.
- validateChoices(move, round, includeInitiator) проверяет выборы для Тумана и Сплетен. Собирает игроков, от которых требуется карта, проверяет ровно по одному допустимому выбору и не даёт повторно выбрать разыгранную карту инициатора.
- validateHound(move) требует MoveDetails.Hound и карту, которая действительно находится у цели.
- validateWerewolf(round) разрешает Оборотня на четвёртом ходу, а на втором/третьем — при двух ранее сыгранных Охотниках. На первом запрещает.
- validateMandragora(round) аналогично проверяет Мандрагору, но условие второго/третьего хода — два сыгранных Мирных жителя.
- ownsAll(player, cards) сравнивает требуемое количество каждого типа карты с количеством в руке. Это важно для повторяющихся одинаковых карт: простой contains мог бы ошибочно разрешить отдать два экземпляра при наличии одного.

Применение эффектов

- applyRumors(round) выбирает случайную карту у каждого игрока, сначала забирает выбранные карты, затем передаёт следующему по кругу. Разделение «сначала забрать, затем раздать» не даёт переданной карте повторно попасть в выбор в том же эффекте.
- applyTheft(move) забирает случайную карту у цели и отдаёт её инициатору, затем передаёт цели заранее выбранную карту инициатора.
- applyGossip(move, round) собирает выбранные карты игроков, удаляет их из рук и раздаёт соседям по кругу согласно индексу в списке игроков.
- applyFog(move) забирает выбранные карты остальных игроков, перемешивает их и

раздаёт этим игрокам обратно.

- `applyTrade(move)` удаляет выбранные карты у двух сторон, затем добавляет каждой стороне карты другой. Метод меняет руки игроков; очередь меняет `Round` после возврата результата.
 - `applyFriend(move, round)` находит у цели Охотника, сбрасывает его и, если колода не пуста, выдаёт взамен новую карту.
 - `applyHunter(move, round)` проверяет номер хода, Укрытие у цели и наличие монстра. Если монстр есть, вызывает его `canBeKilledByHunter(...)`; при успешном убийстве сбрасывает монстра и возвращает `HUNTERS_WON`. Иначе возвращает обычное продолжение.
 - `applyHound(move)` убирает Пса из руки инициатора, заставляет цель сбросить выбранную карту и передаёт Пса цели. Если сброшен монстр, возвращает победу охотников.
- Особый порядок нужен, потому что Пёс переходит в руку, а не в стопку разыгранных.

8. Хранилище и статистика

Договор `GameRepository`

- `registerPlayer(name)` — сохранить имя в реестре до первой партии.
- `writeGame(result)` — сохранить завершённую партию и вернуть её ID.
- `getHistory()` — получить сохранённые партии с участниками и ходами.
- `getStats()` — получить статистику всех зарегистрированных игроков, включая игроков без партий.
- `deleteGame(gameId)` — удалить одну партию и сообщить, была ли она найдена.

Контроллер GUI и `GameAdmin` зависят от этого интерфейса. Они не знают, как устроены таблицы `Exposed`.

`SQLiteGameRepository` — создание объекта и таблиц

Блок `init` превращает путь в абсолютный, создаёт родительский каталог, подключается к `SQLite` через `JDBC` и создаёт таблицы `games`, `players`, `game_players`, `moves` при необходимости. Это инфраструктурный код: он не участвует в проверке игровых правил.

`registerPlayer(name)`

Убирает внешние пробелы, запрещает пустое имя и длину больше 100 символов. В транзакции ищет совпадение без учёта регистра и добавляет запись в `players`, если имени ещё нет. Это реестр игроков, независимый от наличия сыгранных партий.

`writeGame(result)`

В **одной транзакции** добавляет строку партии со временем, связывает игроков с партией и их очками/победой, затем сохраняет журнал ходов с номерами раундов и ходов. Для имени вызывает `findOrCreatePlayer()`. Возвращает ID партии. Одна транзакция нужна, чтобы при сбое не осталась половина результата.

`getHistory()`

Читает партии из таблицы, каждую собирает через `readGame()`, сортирует от нового ID к старому. Внешнему коду отдаёт `StoredGame`, а не строки SQL-таблиц.

`getStats()`

Читает имена всех зарегистрированных игроков и результаты их партий, преобразует строки связующей таблицы в `PlayerGameRecord`, вызывает `StatisticsCalculator.calculate()`. Передача всех имён позволяет включить зарегистрированного игрока с нулём игр.

`deleteGame(gameId)`

В транзакции удаляет ходы и связи игроков с партией, затем саму партию; возвращает `true`, если удалена запись игры. Профили в таблице `players` остаются, статистика при следующем чтении пересчитывается по оставшимся партиям.

`findOrCreatePlayer(name)`

Ищет ID игрока по имени; при отсутствии вставляет новую запись и возвращает новый ID. Метод нужен при сохранении партии, в том числе созданной через консоль без отдельной регистрации. Он приватный, потому что это деталь работы репозитория.

`readGame(row)`

По строке партии подгружает имена и результаты участников, затем ходы. Сортирует участников по UID, группирует ходы по раундам и упорядочивает внутри раунда. Собирает `StoredGame`, удобный для экрана истории и для внешнего кода. Вложенный `StoredMove` — временная структура для сортировки, не отдельная операция.

`StatisticsCalculator.calculate(records, playerNames)`

Группирует результаты партий по имени, для каждого зарегистрированного имени считает число игр, побед и сумму очков. Затем сортирует по победам, общим очкам и имени. Это чистое вычисление без SQLite и Swing: его легко проверить отдельным unit-тестом.

`PlayerStatistics.averageScore` — **вычисляемое свойство**

Это не обычная `fun`, но логика выполняется при чтении свойства: возвращает `0.0`, если игр нет, иначе делит сумму очков на число игр. Среднее не хранится отдельной колонкой и всегда соответствует текущим `gamesPlayed` и `totalScore`.

Таблицы и модели данных

`DatabaseTables.kt` содержит определения схемы, а не функции: `GamesTable` — партии, `PlayersTable` — реестр имён, `GamePlayersTable` — участники и результаты партии, `MovesTable` — журнал. `StoredGame`, `StoredPlayerResult`, `PlayerStatistics`, `PlayerGameRecord` — структуры передачи данных. Они отделяют экран и сценарии от `ResultRow` библиотеки `Exposed`.

9. Каждая тестовая функция

Тесты живут в `src/test/kotlin`. Их функции не вызываются приложением: тестовый фреймворк JUnit вызывает методы с `@Test`. Анонимные `validate/apply` в некоторых тестах — специальные заменители обработчика эффектов, чтобы проверить именно порядок вызовов и сценарий без случайной раздачи.

DeckTest

- `deck has enough cards for eight players`: создаёт `Deck(8)`, снимает 32 карты и убеждается, что колода опустела. Это проверка граничного состава: восьми игрокам нужно по четыре карты.
- `every game starts with exactly four cards per player`: создаёт `Game` с восьмью игроками и проверяет размер каждой руки. Тест проверяет связь `Game` → `Round` → `Deck`, а не только отдельный метод колоды.

RoundTest

- `round returns a base move error instead of printing it`: пытается сыграть Мирного жителя, когда обладатель Розыска обязан сыграть Розыск. Проверяет текстовый отказ и отсутствие записи в журнале. Раунд сообщает об ошибке, оставляя вывод интерфейсу.
- `round records the result without applying card rules`: передаёт `recordMove` уже готовый `MoveResult.Applied(HUNTERS_WON)` и проверяет статус и журнал. Подтверждает, что `Round` записывает исход, а не заново вычисляет эффект карты.

CardEffectHandlerTest

- `handler returns victory without changing round status`: готовит ход Псом против монстра, вызывает `validate` и `apply`, затем проверяет: обработчик вернул `HUNTERS_WON`, но `Round.status` ещё `IN_PROGRESS`. Только вызов `Round.recordMove()` меняет статус. Это проверка границы ответственности между эффектом и раундом.

GameMoveTest

- `game validates applies and records a move`: подставляет обработчик со счётчиками вызовов. Недопустимый базовый ход не должен дойти до обработчика; допустимый должен вызвать проверку и применение по одному разу, завершить игру и появиться в истории. Так проверяется координация внутри `Game.processGameMove()`.
- `handler validation rejects without changing round`: подставляет обработчик, который возвращает ошибку из `validate`. Проверяет сообщение отказа, неизменившуюся очередь, карту в руке и отсутствие вызова `apply`. Проверка правил обязана идти перед изменением состояния.
- `game advances to next player after ordinary move`: подставляет эффект, который возвращает `IN_PROGRESS`. Проверяет `MoveApplied`, продолжающуюся игру и смену текущего игрока. Это путь обычного хода без завершения раунда.

StatisticsCalculatorTest

- `statistics are grouped by player name`: проверяет объединение двух результатов Анны, число побед, сумму и среднее. Статистика нескольких партий относится одному профилю.
- `empty history produces no statistics`: без результатов и списка зарегистрированных имён возвращается пустой список. Это базовый случай чистой функции.
- `registered player without games has zero statistics`: передаёт имя отдельно от результатов и получает профиль с нулями. Нужен для регистрации до первой партии.
- `equal wins are sorted by total score then name`: проверяет порядок при одинаковом числе побед, а также средний счёт. Подтверждает явное правило сортировки, реализованное в `calculate()`.

SQLiteGameRepositoryTest

- `registered players persist without games and keep profile after deletion`: проверяет обрезку пробелов, профиль с нулевыми показателями после повторного открытия файла, запрет повторного/пустого/длинного имени, изменение статистики после партии и сохранение профиля после удаления партии.
- `game history survives reopening repository`: сохраняет `GameResult`, создаёт новый объект репозитория на том же файле и сверяет ID, игроков, очки, победителя и журнал. Это проверка реального сохранения на диске, а не только памяти процесса.
- `statistics combine several games of the same players`: записывает две партии с пересекающимися именами и проверяет объединённые показатели каждого игрока. Проверяет связку таблиц и `StatisticsCalculator`.
- `deleting game removes its moves and updates statistics`: удаляет одну из двух партий, повторно удаляет тот же ID, проверяет историю и статистику, а через JDBC убеждается, что в трёх таблицах не осталось строк удалённой партии.
- `history keeps newest game first and move order inside each round`: сохраняет две партии, одна содержит несколько раундов и ходов; проверяет порядок партий, журнал внутри раундов, победителей и время записи.
- `failed write rolls back game players and moves`: пытается сохранить результат с повторяющимся именем, ожидает ошибку и проверяет пустоту всех таблиц. Затем сохраняет корректную партию. Это проверка транзакции: неудачная запись не оставляет частичные данные.
- Приватная `gameResult(vararg scores)` создаёт одинаково устроенный `GameResult` для нескольких тестов: объекты игроков, карту очков, победителей и пример журнала. Это тестовая фабрика данных, а не функция приложения.

DesktopGameControllerTest

- `gui controller hides hand and persists completed game`: с временной SQLite-базой проверяет запрет хода при скрытой руке, открытие карт, завершение партии, автоматическое сохранение, статистику после повторного открытия базы и возврат в лобби. Обработчик эффекта подменён, чтобы партия предсказуемо закончилась первым допустимым ходом.
- `adding names reuses registered player and removing keeps profile`: проверяет обрезку

имени, запрет дубля в текущем составе, удаление из состава, повторное добавление в другом регистре и сохранение одного профиля в базе. Отделяет состав партии от постоянного реестра.

GameAdminSystemTest

- finished game is saved and shown to user: запускает весь консольный сценарий `GameAdmin.runGame()` со специальным вводом/выводом и временной базой. Проверяет сохранённую партию, показ результата, ID, историю и статистику. Так подтверждается связь сценария, домена и репозитория.
- `AutomaticInput.inputPlayers()` в этом тесте возвращает трёх заранее заданных игроков без терминала. Нужен, чтобы тест не ждал человека.
- `AutomaticInput.inputMove(game)` берёт допустимую карту текущего игрока, при необходимости выбирает цель и составляет `MoveDetails`. Нужен, чтобы пройти реальный цикл `GameAdmin` автоматически.
- `RecordingOutput.showGameResult`, `showGameSaved`, `showHistory`, `showStats` записывают полученные значения в поля для последующих проверок. `showGameState`, `showRoundResult`, `showMoveEvent`, `showError` ничего не делают: в данном тесте их текст не проверяется. Это тестовый двойник `Output`.
- Анонимный `CardEffectHandler.validate/apply` в системном тесте принимает ход и немедленно возвращает победу монстра. Аналогичные подмены в тестах `GameMoveTest` и `DesktopGameControllerTest` дают предсказуемый результат и позволяют проверять конкретный слой, не полагаясь на случайные карты.

10. Как объяснить проект преподавателю за минуту

«У меня две точки входа: консольная и графическая. GUI построен на Swing: `DesktopWindow` показывает данные и принимает нажатия, `DesktopGameController` превращает действия пользователя в ход и отвечает за сохранение. Консольный путь ведёт `GameAdmin` через интерфейсы `Input` и `Output`. Оба пути вызывают один `Game`: он проверяет ход в `Round`, проверяет и применяет эффект в `CardEffectHandler`, записывает успешный ход, считает очки и возвращает результат. Доменные классы не знают, где идёт вывод. `GameRepository` отделяет сценарии от `SQLite`, а `SQLiteGameRepository` реализует сохранение. Экран GUI обновляется вручную через `render()` после событий Swing; поэтому это простая архитектура, близкая к MVC, без автоматической реактивности».

11. Где искать функцию в исходниках

Задача	Файл
Запуск консоли	<code>src/main/kotlin/main.kt</code>
Запуск GUI и общий стиль Swing	<code>src/main/kotlin/game/ui/DesktopMain.kt</code>
Окно, кнопки, вкладки, диалоги	<code>src/main/kotlin/game/ui/DesktopWindow.kt</code>

Задача	Файл
Сценарий GUI и сохранение результата	<code>src/main/kotlin/game/ui/DesktopGameController.kt</code>
Консольные ввод и вывод	<code>src/main/kotlin/game/ui/Console.kt</code>
Сценарий консольной партии	<code>src/main/kotlin/game/logic/GameAdmin.kt</code>
Порядок обработки хода, очки	<code>src/main/kotlin/game/domain/Game.kt</code>
Очередь и журнал одного раунда	<code>src/main/kotlin/game/domain/Round.kt</code>
Правила конкретных карт	<code>src/main/kotlin/game/domain/CardEffectHandler.kt</code>
Руки игроков, колода, данные хода	<code>src/main/kotlin/game/domain/Player.kt, Deck.kt, Move.kt</code>
Реестр, история, SQL	<code>src/main/kotlin/game/data/SqliteGameRepository.kt</code>
Подсчёт статистики	<code>src/main/kotlin/game/data/StatisticsCalculator.kt</code>